

Объектно-ориентированное программирование

Лекция 10: Порождающие паттерны

Factory Method (Фабричный метод). Abstract Factory (Абстрактная фабрика). Builder (Строитель). Prototype (Прототип). Singleton (Одиночка).

Порождающие паттерны проектирования абстрагируют процесс инстанцирования.

Они помогут сделать систему независимой от способа создания, композиции и представления объектов.

Паттерн, порождающий классы, использует наследование, чтобы варьировать инстанцируемый класс, а паттерн, порождающий объекты, делегирует инстанцирование другому объекту



Фабричный метод (factory method)

Определяет интерфейс для создания объекта, но оставляет подклассам решение о том, какой класс инстанцировать. Фабричный метод позволяет классу делегировать инстанцирование подклассам.

Используйте паттерн фабричный метод, когда:

- классу заранее неизвестно, объекты каких классов ему нужно создавать;
- класс спроектирован так, чтобы объекты, которые он создает, специфицировались подклассами;
- класс делегирует свои обязанности одному из нескольких вспомогательных подклассов, и вы планируете локализовать знание о том, какой класс принимает эти обязанности на себя.



PlantUML diagram

```
1 class Dog:
2     def speak(self): return "Гав!"
3
4 class Cat:
5     def speak(self): return "Мяу!"
6
7 def get_pet(pet_type):
8     pets = dict(dog=Dog(), cat=Cat())
9     return pets[pet_type]
10
11 p = get_pet("dog")
12 print(p.speak())
```

- Фабричные методы избавляют проектировщика от необходимости встраивать в код зависящие от приложения классы. Код имеет дело только с интерфейсом класса `Product`, поэтому он может работать с любыми определенными пользователями классами конкретных продуктов.
- Потенциальный недостаток фабричного метода состоит в том, что клиентам, возможно, придется создавать подкласс класса `Creator` для создания лишь одного объекта `ConcreteProduct`. Порождение подклассов оправдано, если клиенту так или иначе приходится создавать подклассы `Creator`, в противном случае клиенту придется иметь дело с дополнительным уровнем подклассов.
- предоставляет подклассам операции-зацепки (`hooks`). Создание объектов внутри класса с помощью фабричного метода всегда оказывается более гибким решением, чем непосредственное создание. Фабричный метод создает в подклассах операции-зацепки для предоставления расширенной версии объекта.

- В примере с документом класс `Document` мог бы определить фабричный метод `CreateFileDialog`, который создает диалоговое окно для выбора файла существующего документа. Подкласс этого класса мог бы определить специализированное для приложения диалоговое окно, заместив этот фабричный метод. В данном случае фабричный метод не является абстрактным, а содержит разумную реализацию по умолчанию;
- соединяет параллельные иерархии. В примерах, фабричные методы вызывались только создателем. Но это совершенно необязательно: клиенты тоже могут применять фабричные методы, особенно при наличии параллельных иерархий классов.



Университет
Сириус
Колледж

Абстрактная фабрика



Предоставляет интерфейс для создания семейств взаимосвязанных или взаимозависимых объектов, не специфицируя их конкретных классов.

Используется, когда:

- система не должна зависеть от того, как создаются, компонируются и представляются входящие в нее объекты;
- входящие в семейство взаимосвязанные объекты должны использоваться вместе и вам необходимо обеспечить выполнение этого ограничения;
- система должна конфигурироваться одним из семейств составляющих ее объектов;
- вы хотите предоставить библиотеку объектов, раскрывая только их интерфейсы, но не реализацию.



Университет
Сириус
Колледж

Абстрактная фабрика

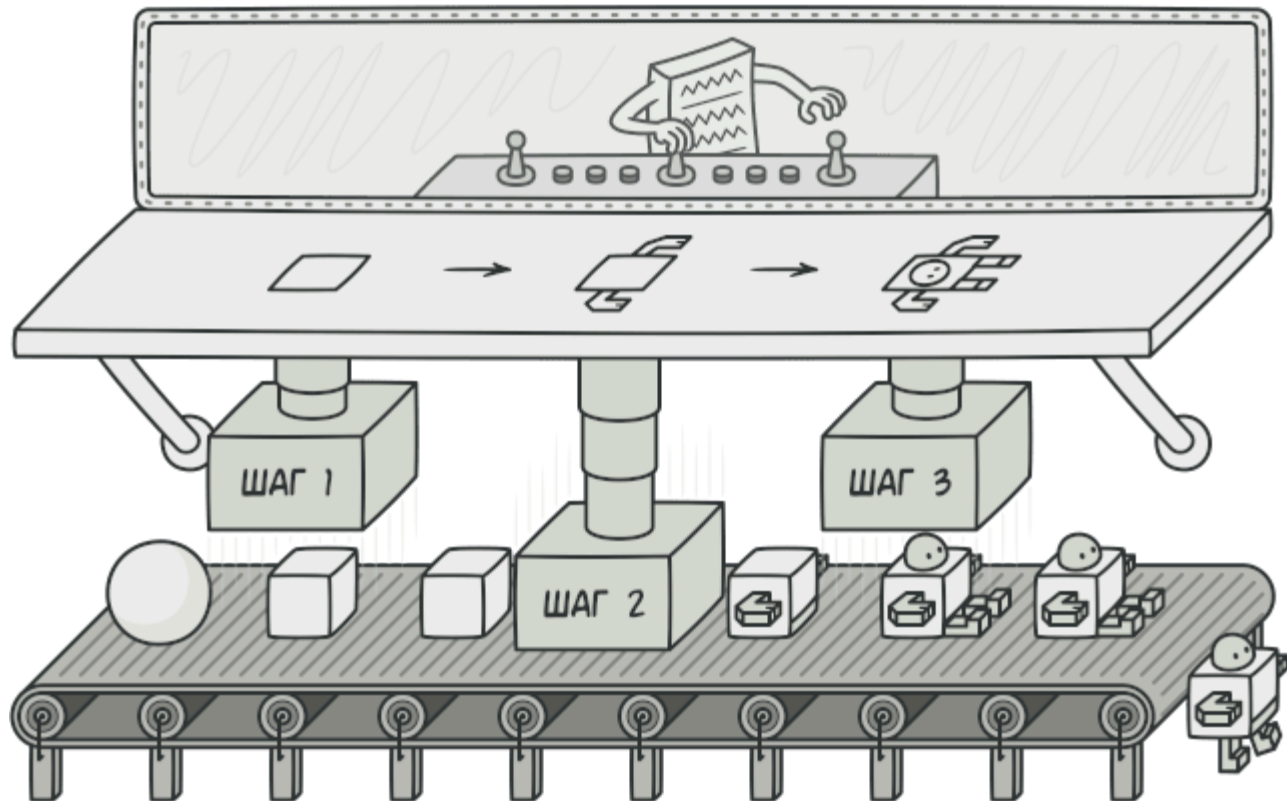


PlantUML diagram

- изолирует конкретные классы. Помогает контролировать классы объектов, создаваемых приложением. Поскольку фабрика инкапсулирует ответственность за создание классов и сам процесс их создания, то она изолирует клиента от деталей реализации классов. Манипулирование экземплярами производится через их абстрактные интерфейсы.
- упрощает замену семейств продуктов. Класс конкретной фабрики появляется в приложении только один раз: при инстанцировании. Это облегчает замену используемой приложением конкретной фабрики. Приложение может изменить конфигурацию продуктов, просто подставив новую конкретную фабрику. Поскольку абстрактная фабрика создает все семейство продуктов, то и заменяется сразу все семейство.
- гарантирует сочетаемость продуктов. Если продукты некоторого семейства спроектированы для совместного использования, то важно, чтобы приложение в каждый момент времени работало только с продуктами единственного семейства. Класс `AbstractFactory` позволяет легко соблюсти это ограничение;



- поддержать новый вид продуктов трудно. Расширение абстрактной фабрики для изготовления новых видов продуктов - непростая задача. Интерфейс `AbstractFactory` фиксирует набор продуктов, которые можно создать. Для поддержки новых продуктов необходимо расширить интерфейс фабрики, то есть изменить класс `AbstractFactory` и все его подклассы.



Отделяет конструирование сложного объекта от его представления, так что в результате одного и того же процесса конструирования могут получаться разные представления.

Используйте паттерн строитель, когда:

- алгоритм создания сложного объекта не должен зависеть от того, из каких частей состоит объект и как они стыкуются между собой;
- процесс конструирования должен обеспечивать различные представления конструируемого объекта.



PlantUML diagram

```
2     def __init__(self):
3         self.pizza = []
4
5     def add_cheese(self):
6         self.pizza.append("cheese")
7         return self # Возвращаем self для цепочки
8
9     def add_pepperoni(self):
10        self.pizza.append("pepperoni")
11        return self
12
13    def build(self):
14        return f"Pizza with {'', '.join(self.pizza)}"
15
16 my_lunch = PizzaBuilder().add_cheese().add_pepperoni().build()
```

```
1  class PizzaBuilder:
2      def __init__(self):
3          self.pizza = []
4
5      def add_cheese(self):
6          self.pizza.append("cheese")
7          return self # Возвращаем self для цепочки
8
9      def add_pepperoni(self):
10         self.pizza.append("pepperoni")
11         return self
12
13     def build(self):
14         return f"Pizza with {'', '.join(self.pizza)}"
15
```

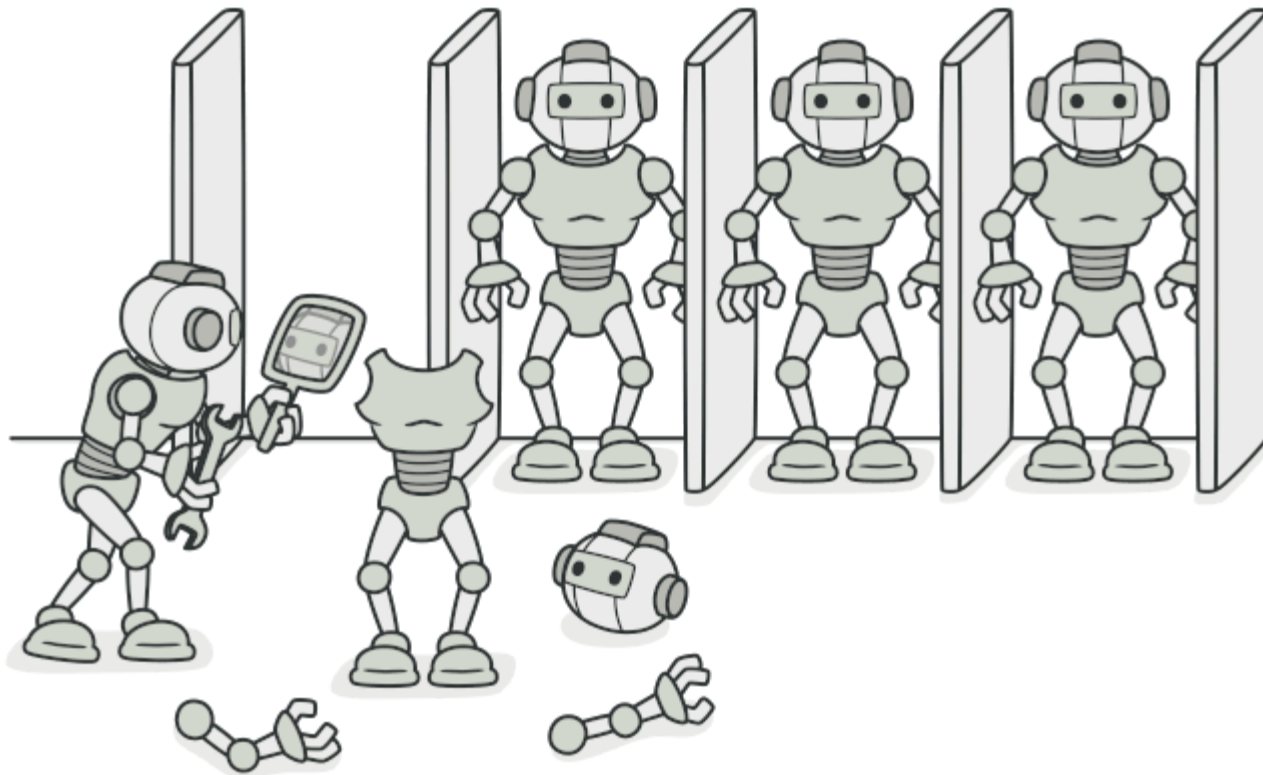
```
2     def __init__(self):
3         self.pizza = []
4
5     def add_cheese(self):
6         self.pizza.append("cheese")
7         return self # Возвращаем self для цепочки
8
9     def add_pepperoni(self):
10        self.pizza.append("pepperoni")
11        return self
12
13    def build(self):
14        return f"Pizza with {'', '.join(self.pizza)}"
15
16 my_lunch = PizzaBuilder().add_cheese().add_pepperoni().build()
```

```
2     def __init__(self):
3         self.pizza = []
4
5     def add_cheese(self):
6         self.pizza.append("cheese")
7         return self # Возвращаем self для цепочки
8
9     def add_pepperoni(self):
10        self.pizza.append("pepperoni")
11        return self
12
13    def build(self):
14        return f"Pizza with {'', '.join(self.pizza)}"
15
16 my_lunch = PizzaBuilder().add_cheese().add_pepperoni().build()
```

```
2     def __init__(self):
3         self.pizza = []
4
5     def add_cheese(self):
6         self.pizza.append("cheese")
7         return self # Возвращаем self для цепочки
8
9     def add_pepperoni(self):
10        self.pizza.append("pepperoni")
11        return self
12
13    def build(self):
14        return f"Pizza with {' '.join(self.pizza)}"
15
16 my_lunch = PizzaBuilder().add_cheese().add_pepperoni().build()
```

- позволяет изменять внутреннее представление продукта. Объект Builder предоставляет распорядителю абстрактный интерфейс для конструирования продукта, за которым он может скрыть представление и внутреннюю структуру продукта, а также процесс его сборки. Поскольку продукт конструируется через абстрактный интерфейс, то для изменения внутреннего представления достаточно всего лишь определить новый вид строителя;
- изолирует код, реализующий конструирование и представление. Паттерн строитель улучшает модульность, инкапсулируя способ конструирования и представления сложного объекта. Клиентам ничего не надо знать о классах, определяющих внутреннюю структуру продукта, они отсутствуют в интерфейсе строителя. Каждый конкретный строитель ConcreteBuilder содержит весь код, необходимый для создания и сборки конкретного вида продукта. Код пишется только один раз, после чего разные распорядители могут использовать его повторно для построения вариантов продукта из одних и тех же частей;

- дает более тонкий контроль над процессом конструирования. В отличие от порождающих паттернов, которые сразу конструируют весь объект целиком, строитель делает это шаг за шагом под управлением распорядителя. И лишь когда продукт завершен, распорядитель забирает его у строителя. Поэтому интерфейс строителя в большей степени отражает процесс конструирования продукта, нежели другие порождающие паттерны. Это позволяет обеспечить более тонкий контроль над процессом конструирования, а значит, и над внутренней структурой готового продукта.



Задаёт виды создаваемых объектов с помощью экземпляра-прототипа и создаёт новые объекты путем копирования этого прототипа.

Используйте Прототип, когда:

- система не должна зависеть от того, как в ней создаются, компонуются и представляются продукты;
- инстанцируемые классы определяются во время выполнения, например с помощью динамической загрузки;
- для того чтобы избежать построения иерархий классов или фабрик, параллельных иерархии классов продуктов;
- экземпляры класса могут находиться в одном из не очень большого числа различных состояний. Может оказаться удобнее установить соответствующее число прототипов и клонировать их, а не инстанцировать каждый раз класс вручную в подходящем состоянии.



Университет
Сириус
Колледж

Прототип (Prototype)



PlantUML diagram



```
1  import copy
2  from typing import List
3
4  class Disk:
5      """Вспомогательный класс: Жесткий диск"""
6      def __init__(self, size_gb: int, content: List[str]):
7          self.size_gb = size_gb
8          self.content = content
9
10 class VirtualMachine:
11     """Сложный объект, который мы будем клонировать"""
12     def __init__(self, name: str, cpu: int, ram: int, disk: Disk, ip: str):
13         self.name = name
14         self.cpu = cpu
15         self.ram = ram
```



```
1  import copy
2  from typing import List
3
4  class Disk:
5      """Вспомогательный класс: Жесткий диск"""
6      def __init__(self, size_gb: int, content: List[str]):
7          self.size_gb = size_gb
8          self.content = content
9
10 class VirtualMachine:
11     """Сложный объект, который мы будем клонировать"""
12     def __init__(self, name: str, cpu: int, ram: int, disk: Disk, ip: str):
13         self.name = name
14         self.cpu = cpu
15         self.ram = ram
```



```
25         # При клонировании мы можем сразу сбросить уникальные данные
26         new_vm.name += "_clone"
27         new_vm.ip = "0.0.0.0" # Требует новой настройки
28         return new_vm
29
30
31 # 1. Долгая и сложная настройка "Золотого образа"
32 boot_disk = Disk(size_gb=50, content=["OS_Linux", "App_Server", "Configs"])
33 golden_image = VirtualMachine(
34     "Ubuntu-Base",
35     cpu=4,
36     ram=16,
37     disk=boot_disk,
38     ip="192.168.1.10",
39 )
```



```
7         self.size_gb = size_gb
8         self.content = content
9
10 class VirtualMachine:
11     """Сложный объект, который мы будем клонировать"""
12     def __init__(self, name: str, cpu: int, ram: int, disk: Disk, ip: str):
13         self.name = name
14         self.cpu = cpu
15         self.ram = ram
16         self.disk = disk
17         self.ip = ip
18
19     def clone(self):
20         """
21         Создает полную копию (deep copy) виртуальной машины.
```



```
28         return new_vm
29
30
31 # 1. Долгая и сложная настройка "Золотого образа"
32 boot_disk = Disk(size_gb=50, content=["OS_Linux", "App_Server", "Configs"])
33 golden_image = VirtualMachine(
34     "Ubuntu-Base",
35     cpu=4,
36     ram=16,
37     disk=boot_disk,
38     ip="192.168.1.10",
39 )
40
41 # 2. Массовое развертывание (Клонирование)
42 # Нам не нужно заново указывать CPU, RAM и содержимое диска
```



```
17         self.ip = ip
18
19     def clone(self):
20         """
21         Создает полную копию (deep copy) виртуальной машины.
22         Важно использовать deepcopy, так как Disk – это изменяемый объект.
23         """
24         new_vm = copy.deepcopy(self)
25         # При клонировании мы можем сразу сбросить уникальные данные
26         new_vm.name += "_clone"
27         new_vm.ip = "0.0.0.0" # Требуется новая настройка
28         return new_vm
29
30
31 # 1. Долгая и сложная настройка "Золотого образа"
```



Прототип (Prototype)

```
17         self.ip = ip
18
19     def clone(self):
20         """
21         Создает полную копию (deep copy) виртуальной машины.
22         Важно использовать deepcopy, так как Disk – это изменяемый объект.
23         """
24         new_vm = copy.deepcopy(self)
25         # При клонировании мы можем сразу сбросить уникальные данные
26         new_vm.name += "_clone"
27         new_vm.ip = "0.0.0.0" # Требуется новая настройка
28         return new_vm
29
30
31 # 1. Долгая и сложная настройка "Золотого образа"
```



```
19     def clone(self):
20         """
21         Создает полную копию (deep copy) виртуальной машины.
22         Важно использовать deepcopy, так как Disk – это изменяемый объект.
23         """
24         new_vm = copy.deepcopy(self)
25         # При клонировании мы можем сразу сбросить уникальные данные
26         new_vm.name += "_clone"
27         new_vm.ip = "0.0.0.0" # Требуется новая настройка
28         return new_vm
29
30
31 # 1. Долгая и сложная настройка "Золотого образа"
32 boot_disk = Disk(size_gb=50, content=["OS_Linux", "App_Server", "Configs"])
33 golden_image = VirtualMachine(
```



Прототип (Prototype)

```
21         Создает полную копию (deep copy) виртуальной машины.
22         Важно использовать deepcopy, так как Disk – это изменяемый объект.
23         """
24         new_vm = copy.deepcopy(self)
25         # При клонировании мы можем сразу сбросить уникальные данные
26         new_vm.name += "_clone"
27         new_vm.ip = "0.0.0.0" # Требуется новая настройка
28         return new_vm
29
30
31 # 1. Долгая и сложная настройка "Золотого образа"
32 boot_disk = Disk(size_gb=50, content=["OS_Linux", "App_Server", "Configs"])
33 golden_image = VirtualMachine(
34     "Ubuntu-Base",
35     cpu=4,
```



```
37         disk=boot_disk,  
38         ip="192.168.1.10",  
39     )  
40  
41 # 2. Массовое развертывание (Клонирование)  
42 # Нам не нужно заново указывать CPU, RAM и содержимое диска  
43 worker_node_1 = golden_image.clone()  
44 worker_node_1.name = "Worker-01"  
45 worker_node_1.ip = "192.168.1.101"  
46 # Изменим диск клона, чтобы доказать независимость объектов  
47 worker_node_1.disk.content.append("Worker_Logs.txt")  
48  
49 worker_node_2 = golden_image.clone()  
50 worker_node_2.name = "Worker-02"  
51 worker_node_2.ip = "192.168.1.102"
```

У прототипа те же самые результаты, что у абстрактной фабрики и строителя: он скрывает от клиента конкретные классы продуктов, уменьшая тем самым число известных клиенту имен. Кроме того, все эти паттерны позволяют клиентам работать со специфичными для приложения классами без модификаций.

- добавление и удаление продуктов во время выполнения. Прототип позволяет включать новый конкретный класс продуктов в систему, просто сообщив
- клиенту о новом экземпляре-прототипе. Это несколько более гибкое решение по сравнению с тем, что удастся сделать с помощью других порождающих паттернов, ибо клиент может устанавливать и удалять прототипы во время выполнения;



- спецификация новых объектов путем изменения значений. Динамические системы позволяют определять поведение за счет композиции объектов - например, путем задания значений переменных объекта, - а не с помощью определения новых классов. По сути дела, вы определяете новые виды объектов, инстанцируя уже существующие классы и регистрируя их-экземпляры как прототипы клиентских объектов. Клиент может изменить поведение, делегируя свои обязанности прототипу. Такой дизайн позволяет пользователям определять новые классы без программирования. Фактически клонирование объекта аналогично инстанцированию класса. Паттерн прототип может резко уменьшить число необходимых системе классов;

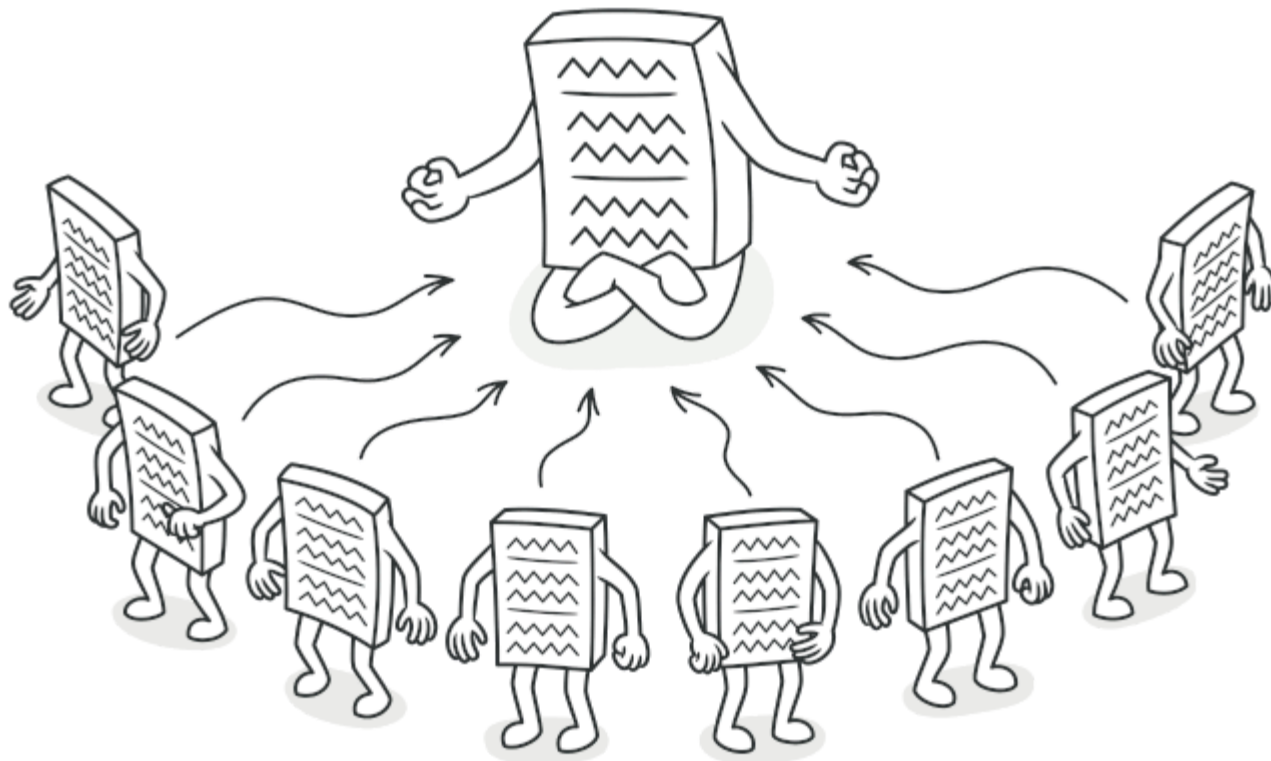
- специфицирование новых объектов путем изменения структуры. Многие приложения строят объекты из крупных и мелких составляющих. Например, редакторы для проектирования печатных плат создают электрические схемы из подсхем. Такие приложения часто позволяют инстанцировать сложные, определенные пользователем структуры, скажем, для многократного использования некоторой подсхемы. Паттерн прототип поддерживает и такую возможность. Мы просто добавляем подсхему как прототип в палитру доступных элементов схемы. При условии, что объект, представляющий составную схему, реализует операцию `Clone` как глубокое копирование, схемы с разными структурами могут выступать в качестве прототипов;
- уменьшение числа подклассов. Прототип позволяет клонировать прототип, а не запрашивать фабричный метод создать новый объект;
- динамическое конфигурирование приложения классами. Некоторые среды позволяют динамически загружать классы в приложение во время его выполнения.

Основной недостаток паттерна прототип заключается в том, что каждый подкласс класса `Prototype` должен реализовывать операцию `Clone`, а это далеко не всегда просто. Например, сложно добавить операцию `Clone`, когда рассматриваемые классы уже существуют. Проблемы возникают и в случае, если во внутреннем представлении объекта есть другие объекты или циклические ссылки.



Университет
Сириус
Колледж

Одиночка (Singleton)



Гарантирует, что у класса есть только один экземпляр, и предоставляет к нему глобальную точку доступа.

Используйте паттерн синглтон, когда:

- должен быть ровно один экземпляр некоторого класса, легко доступный всем клиентам;
- единственный экземпляр должен расширяться путем порождения подклассов, и клиентам нужно иметь возможность работать с расширенным экземпляром без модификации своего кода.

У паттерна Singleton есть определенные достоинства:

- контролируемый доступ к единственному экземпляру. Поскольку класс Singleton инкапсулирует свой единственный экземпляр, он полностью
- контролирует то, как и когда клиенты получают доступ к нему;
- уменьшение числа имен. Паттерн Singleton - шаг вперед по сравнению с глобальными переменными. Он позволяет избежать засорения пространства имен глобальными переменными, в которых хранятся уникальные экземпляры;
- допускает уточнение операций и представления. От класса Singleton можно порождать подклассы, а приложение легко сконфигурировать экземпляром расширенного класса. Можно конкретизировать приложение экземпляром того класса, который необходим во время выполнения;



Одиночка (Singleton)

- допускает переменное число экземпляров. Паттерн позволяет вам легко изменить свое решение и разрешить появление более одного экземпляра класса `Singleton`. Вы можете применять один и тот же подход для управления числом экземпляров, используемых в приложении. Изменить нужно будет лишь операцию, дающую доступ к экземпляру класса `Singleton`;
- большая гибкость, чем у операций класса. Еще один способ реализовать функциональность одиночки - использовать операции класса, то есть статические функции-члены (`classmethod`) препятствуют изменению дизайна, если потребуется разрешить наличие нескольких экземпляров класса.



Одиночка (Singleton)

```
1  class Database:
2      _instance = None
3
4      def __new__(cls):
5          if cls._instance is None:
6              cls._instance = super().__new__(cls)
7              print("Подключение к БД создано")
8              return cls._instance
9
10 d1 = Database()
11 d2 = Database()
12 print(d1 is d2)  # True
```